

IHK-Training – BubbleSort · Lineare Suche · Binäre Suche

Druckfreundlich · Java-ähnlicher Pseudocode · kompakt und prüfungsorientiert

Prüfungstipp: Schreib kurz und klar: Initialisieren → Schleife(n) → Bedingung → Aktion → Rückgabe.

1) BubbleSort (stabil, $O(n^2)$)

Vergleicht benachbarte Elemente und vertauscht sie, wenn sie falsch stehen. Nach Durchlauf i sind die i größten Elemente rechts „eingebubbelt“.

```
void bubbleSort(int[] a) {
    for (int i = 0; i < a.length - 1; i++) {
        for (int j = 0; j < a.length - 1 - i; j++) {
            if (a[j] > a[j + 1]) {
                int tmp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = tmp;
            }
        }
    }
}
```

Tipp: Erkennbar an „benachbarte Elemente vertauschen“. Prüfe die Grenzen: $j < a.length - 1 - i$.

2) Lineare Suche ($O(n)$)

Prüft alle Elemente der Reihe nach. Funktioniert ohne Sortierung.

```
int linearSearch(int[] a, int x) {
    for (int i = 0; i < a.length; i++) {
        if (a[i] == x) return i;
    }
}
```

```
    return -1;
}
```

Tipp: Sobald gefunden → return. Sonst am Ende -1.

3) Binäre Suche ($O(\log n)$, nur sortiert)

Halbiert den Suchbereich jedes Mal. Voraussetzung: **aufsteigend sortiertes** Array.

```
int binarySearch(int[] a, int x) {
    int left = 0, right = a.length - 1;
    while (left <= right) {
        int mid = (left + right) / 2; // Ganzzahldivision
        if (a[mid] == x) return mid;
        if (a[mid] < x) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}
```

Tipp: Achte im Text auf Worte wie „sortiert“, „halbieren“, „mittleres Element“.

4) Mini-Übungen (IHK-Stil)

A) BubbleSort – Handsimulation

Gegeben: $a = \{5, 2, 4, 3\}$. Notiere das Array nach jedem äußeren Durchlauf ($i = 0..2$).

- $i=0 \rightarrow ?$
- $i=1 \rightarrow ?$
- $i=2 \rightarrow ?$

B) Binäre Suche – mid-Werte

Gegeben (sortiert): $a = \{2, 4, 7, 9, 12, 15\}$. Suche $x = 9$. Welche mid-Indizes entstehen?

- Schritt 1: left=0, right=5 → mid=?
- Schritt 2: ...

5) Vergleichstabelle

Verfahren	Idee	Voraussetzung	Richt-Laufzeit	Stabil
BubbleSort	Nachbarn vergleichen/tauschen	–	$O(n^2)$	ja
Lineare Suche	Alle prüfen	–	$O(n)$	–
Binäre Suche	Bereich halbieren	sortiert	$O(\log n)$	–